Leonzilla

version 1.11.19

Uploads your projects from the PC to the web with one click – over FTP, FTPS or SFTP. It also builds, backs up to ZIP and pushes to GitHub. It never deletes anything on the server.

- passwords protected by a master password
- portable – nothing to install
- only uploads, never deletes

This guide is also built into the app – in Leonzilla click the **?** button in the top bar (next to the settings) or press **F1**.

CONTENTS

01 Quick start
02 Buttons
03 Exclusions and patterns
04 What gets uploaded
05 Build
06 Backups
07 GitHub
08 FW build and monitor
09 Passwords and security
10 Portable and more PCs
11 Keys and tricks

Quick start

1. **New card** – the `+` button in the bar or `Ctrl`+`T`. Every project (website, firmware...) has its own card.
2. **Fill in what the card should do** – any of these is enough: *synchronization to a server* (server + folder pairs PC → server), *build*, *backup*, *GitHub*, *FW build*. Everything is saved immediately. The *Synchronize* button appears once both a server and a folder are filled in; a backup-only card leaves the folder section empty.
3. **Settings tabs.** In *Edit card* the settings are split into tabs *Synchronization* (folders, server, check, exclusions), *Own script*, *Backups*, *GitHub*, *FW ESP32* and finally *Build (RAD Studio)*. *App settings* are split the same way: *General* (theme, language, start with Windows, parallel uploads, master password, export / import, more PCs), *Groups* and *Builds (RAD Studio)*.

4. **Save.** Card changes are written only with *Save* – then the card is locked and only shows its settings (it cannot be changed by accident). *Close without saving* puts the card back as it was. Switching to another card keeps unfinished changes (● on the tab); closing the tab asks. Change it again with *Edit card*.

5. **Overview** – all cards as tiles, sorted into groups (Settings → Groups). Drag a tile to move it to another group. Click a group name to collapse it – each computer remembers its own collapsed groups (and open tabs).

6. **Tabs at the top** = open cards only. Open a card by clicking its tile, close it with ✕ on the tab (or `Ctrl` + `W`, middle mouse button) – it stays in the Overview. When the tabs do not fit, arrows ‹ › appear (the mouse wheel scrolls too).

A similar project? **Duplicate** copies the whole card including the password – then just change what differs. Delete a card you no longer need in *Edit card* → *Delete* or with a right click on its tab or tile (files on the PC and on the server are not touched).

Buttons

↑ Synchronize

Compares the folders with the server and uploads new and changed files. You confirm before it starts (and tick the checklist items, if any are set).

Build + Synchronize

Compiles the selected builds first and uploads only when all of them succeed. *Build only* / *Upload only* does just one part.

Check

A preview without uploading – shows what would be uploaded.

Back up

Packs the configured folders into ZIP files with the date and time in the name.

GitHub

Goes through the card's repositories, shows the changes and commits and pushes.

Duplicate

A new card with the same settings, right after the original.

On the tile of a card that **synchronizes to a server**, *Check* and *Synchronize* are the main buttons; backup, GitHub or FW are small chips below them. On a card **without synchronization** all actions (script, FW, Monitor, Back up, GitHub) are equal – buttons of the same size, two side by side; a button with a long label or an odd last one takes the whole row.

Exclusions and patterns

What should not be uploaded or backed up. Exclusions can be set for each folder pair / backup (buttons *folder*, *file*, *pattern*) and for the whole card in *Exclude in all folders* (or *in all backups*) – one rule per line.

PATTERN	EXCLUDES	EXAMPLE
`.git/`	a folder with this name **anywhere**	`.git/` `managed_components/`
`build*/`	folders whose name **starts** with build – anywhere	build, build_dev, build_prod
`*.psd`	files by mask – anywhere	`*.tmp` `~$*`
`img/raw/`	one **specific** folder (a slash in the middle = path from the source folder)	only img/raw, not other raw
`firmware/esp32/build*/`	a mask in the given path only	build folders only in firmware/esp32
`docs/manual.pdf`	one specific file	
`photo?.jpg`	`?` = exactly one character	photo1.jpg, photoA.jpg
`# note`	a line starting with # is ignored	

- **A trailing slash** = folders only (a file with the same name stays).
- `*` does not cross a slash: `a/*/` matches `a/x/`, not `a/x/y/`. A pattern without a slash in the middle applies on every level.
- Letter case does not matter, `\` and `/` mean the same.
- An excluded folder is not walked at all – this makes the comparison faster.

What gets uploaded

- The file is **missing** on the server or has a **different size** → it is uploaded.
- The file on the PC **changed since the last upload** (Leonzilla remembers the size and time) → it is uploaded even with the same size.
- **Nothing is deleted** on the server – files deleted on the PC stay there.
- Files are uploaded through a temporary file that is renamed when complete (the website never sees a half-uploaded file), and the size is verified.
- To be sure, tick *Upload all files again* in the confirmation.

Build (RAD Studio / TMS WEB Core)

- Builds are created once in *App settings* → *Builds* (a `.dproj` project, configuration, switches); a card just selects and orders them.
- Switches are read from the top part of `Predefines.inc` in a `{$IFNDEF LEONZILLA} … {$ENDIF}` block: RAD Studio uses the file as it is, a Leonzilla build skips the block and passes `LEONZILLA` + the switches ticked in the build. When a new one appears, load it with *Load switches*.
- Optionally line endings are converted to CRLF and the project's post-build event runs. The output (TMSWeb) is excluded from Dropbox so that Dropbox does not lock the files.
- **Dropbox during build** (Edit card → *Dropbox*, for own script, FW ESP32 and RAD Studio; default *don't watch*): Dropbox touches files even in ignored folders and the build can then fail with "file is being used by another process". *Check whether it is paused* – before the build Leonzilla reads the tooltip of the Dropbox tray icon; if it is not paused, it asks (stop it for the build – back within seconds – or pause it). *Always stop for the build* – stops it without asking and starts it again after the build.
- **Switches for one build only**: in the build confirmation click a switch to *strike it out* (click again to put it back) and use *+ switch* to add another one (it offers the known switches of the build, a new one can be typed). It applies to this one build only – the build setting is not changed and next time the original switches are back. The change is shown in the build log too.
- **Version info** (in the build settings, 1 line, max 80 characters, default = build name): the post-build and the command after build get it in the `LEONZILLA_VERSION_INFO` variable – e.g. to write it into `version.json`. If your project's script reads a different variable name, add it in *App settings* → *Builds*. Empty = no info. The text can be changed in the build confirmation – **the card remembers it** and offers it next time; ↺ restores the default from the build settings.
- The progress window shows the steps and the compiler output; *Hide* keeps the build running in the background.
- **Own script** (a section in the card settings; the card then says "Script" instead of "Build"): instead of a RAD Studio project just a **path to a script** – Python (`.py`), batch (`.bat`, `.cmd`) or PowerShell (`.ps1`) – and optional parameters. The script runs in its own folder (Python from PATH, otherwise the `py` launcher); when it ends without an error (exit code 0), the card uploads the folders to the server. On the card it looks and works like a build – *Build + Synchronize*, *Build only*, *Upload only*. When the card also has RAD Studio builds, Build light runs after them. An own **button label** (e.g. *Build and upload to the web*) replaces the default text – handy for a script that uploads by itself (such as publishing a program on a website); such a card needs no server or folders. **More scripts** (button *Add script*): the card then shows *Run scripts* and in the confirmation you choose **which one** script runs; the card remembers the choice. With more scripts the label field is the *name in the choice*. **More sets of parameters** (button *Add parameters* at a script; a set may be empty = no parameters): in the confirmation you choose which parameters the script runs with (remembered). When you remove scripts and parameters down to one, the card works as before, including its own button label – existing cards are not changed.

Backups

- Each backup = folder → target folder. It creates `prefix_YYYY-MM-DD_hh-mm-ss.zip` (empty prefix = folder name).
- ZIP name in the confirmation – for each backup separately, with the *Note* | *Own name* switch: empty field = default `web_2026-10-18_10-41-14.zip` · *Note* = inserted between the name and the date `web_new cfg_2026-10-18_10-41-14.zip` · *Own name* = exactly your name without a date `fw-160.zip` (if it exists, `fw-160 (2).zip` is created – nothing is overwritten). You see the resulting name immediately; `Enter` jumps to the next field, in the last one it starts the backup.
- When a card has more backups, each one has a **checkbox** in the confirmation (ticked by default). An unticked backup is skipped this time – the card settings do not change.
- The name fields can be turned off for the whole card: *Edit card → Backups → Offer a note or own ZIP name when backing up*. The confirmation then only shows the default ZIP names.
- Exclusions work as with synchronization – typically `.git/`, `build*/`, `managed_components/`.
- The ZIP is written as `.zip.part` and renamed only when complete. A locked file is skipped with a warning. Leonzilla never deletes old backups.

GitHub

- Select the folders with git repositories in the card. Each repository knows where it belongs (`origin`).
- The GitHub button goes through them one by one and **skips those without changes**. Otherwise it shows the changed files (click = changes in the file), a commit message and the buttons *Commit + Push*, *Commit only*, *Skip*.
- **Only ticked files** are committed. Push goes to the current branch (usually `main`).
- *Pull changes before push* – when you also commit from another PC.
- Needs **Git for Windows**. Git handles the sign-in itself – Leonzilla stores no token.

FW build and monitor (ESP-IDF / ESP32)

- **Monitor** (a checkbox in the *FW build and monitor* section): the card and its tile get a *Monitor* button – pick a COM port, *Open monitor* and it runs, even without a build or flashing. The monitor alone needs no project folder.
- In the card's *FW build* section: the project folder (with `CMakeLists.txt`), the ESP-IDF activation (PowerShell profile from EIM – empty = found in `C:\Espressif\tools`) and **variants**. The *Load from tools\build_*.cmd* button creates them (name, build folder from `-B`, PROD scripts as "no flashing").

- The **FW build** button: pick a variant, an action (*Build*, *Build + Flash*, *Build + Flash + Monitor*, *Monitor only*) and a COM port. The choice is remembered per card.
- **Clean config** is ticked at start time (every time again) – it deletes `<build folder>\sdkconfig`, which is then recreated from `sdkconfig.defaults`. Needed after changing `sdkconfig.defaults`, the profile or Kconfig.
- **Reconfigure** (ticked by default, again each time the window opens): runs `tools\build_dev.cmd reconfigure` before the build (reloads CMake / sdkconfig).
- The **monitor** is right in the window: a coloured device output (errors red, warnings yellow, `@…` replies blue) and a command line at the bottom – `Enter` sends, `↑` `↓` history, *Device reset* = restart via RTS. Select text in the output with the mouse and copy it with `Ctrl`+`C`; top right *Copy all* and *Clear*. The output does not scroll while you select. *Hide* keeps the monitor running (the tile shows "FW monitor running").
- *ESP-IDF terminal* opens a normal PowerShell with ESP-IDF active in the project folder – for `menuconfig` and other commands.
- No other program (e.g. a VS Code monitor) may have the port open – Leonzilla tells you.

Passwords and security

Without a master password

Server passwords in `config.json` are **obfuscated** (AES with a key inside the app). They are not readable in the file, but whoever has the file and Leonzilla can get them.

With a master password

Passwords are **encrypted** – the key is derived from the master password (scrypt) and encrypted with AES-256-GCM. Without the master password nobody can read them from `config.json`. Turn it on in *App settings*; Leonzilla asks for it at start.

- **A forgotten master password cannot be recovered** – you then enter the server passwords again.
- SFTP: the server key is remembered on the first connection; when it changes, Leonzilla refuses to connect and asks.
- The window talks only to Leonzilla on this PC (`127.0.0.1`, a random port and token). Nothing is sent anywhere else.

Portable and more PCs

- Nothing to install: `Leonzilla.exe` and the `Leonzilla_data` folder next to it (cards, settings, upload state, logs). All of it can live in Dropbox and run on several PCs.
- Only the window size and position and the window cache are kept per PC (outside Dropbox, in `%LOCALAPPDATA%\Leonzilla`).

- Changes from another PC are loaded when you return to the window. Do not synchronize the same card on two PCs at the same time.
- One Leonzilla runs per PC – starting it again switches to the open window.
- **Start with Windows** (App settings → Start, off by default): Leonzilla starts minimized in the taskbar after you sign in. Set per PC.
- Needs Windows 10/11 with WebView2 (always present in Windows 11).
- **Export / import.** *App settings → General → Export* saves cards, groups, builds and general settings into one file (default folder `Leonzilla_data\export`) – for a backup or moving to another installation. Server passwords only when you tick *Including server passwords*, then encrypted with an export password. *Import…* shows what the file contains and only **adds** the ticked cards, groups and builds – it overwrites nothing (a group or build with the same name is reused) and backs up `config.json` to `Leonzilla_data\backup` first. A tab that has something set is green with a dot and a short summary (e.g. "2 scripts"); tabs with nothing set are grey. The arrow keys ← → switch between the tabs.
- **Language** – Czech or English: *App settings → Language* (default by the Windows display language).

Keys and tricks

`Ctrl` + `T` new card	`Ctrl` + `Tab` next card (`Shift` = previous)
`Ctrl` + `Enter` in a commit message = Commit + Push	`Esc` closes a window / dialog
`Ctrl` + `W` closes the card tab (the card stays in the Overview)	`right click` on a tab or tile: edit, duplicate, colour, delete (the colour can also be changed at the top of the card editor)
`double click` on the tab of a card being edited = rename	

- Tabs and tiles can be dragged (order, groups).
- **Click the logo** top left = *About* (version, links to GitHub and the latest version, bug reports).
- The *Log* button opens the card's full log; after a crash the output is in `Leonzilla_data\logs\crash.log`.



Thank you for using Leonzilla.
Enjoy it!

© 2026 Leoš Vilím · leotech.cz · report a bug / idea

☕ Support development – buy the author a coffee